TP 4 — API Temps Réel et Événementielles

Ralph Bou Nader

Master — Ingénierie Informatique

Objectifs

Ce TP couvre le Chapitre 6 : vous allez manipuler un vrai canal **WebSocket**, observer un flux **Server-Sent Events** réel, construire votre propre endpoint SSE avec Spring Boot, inspecter un **Webhook** en temps réel, puis concevoir une architecture **pub/sub** sur papier.

Prérequis

- Postman (version récente : l'onglet **New → WebSocket** doit être disponible).
- Un navigateur web.
- Le projet Spring Boot du TP2 (ou un nouveau projet avec la dépendance `spring-boot-starter-web`).
- Aucune installation de serveur Kafka n'est nécessaire pour ce TP.

Partie 1 — WebSockets

Tâche 1 — Se connecter à un serveur WebSocket public

Nous utilisons le serveur d'écho public fourni par Postman lui-même, conçu pour ce type d'exercice.

1. Dans Postman, cliquez sur **New → WebSocket Request**.
2. Entrez l'URL : `wss://ws.postman-echo.com/raw`.
3. Cliquez sur **Connect**.

Question à consigner : quel code de statut apparaît lors de la connexion ? Faites le lien avec le *handshake* vu en cours (Section 3.2 du Chapitre 6) — quel code HTTP confirme la bascule de protocole ?

Tâche 2 — Envoyer et recevoir des messages sur une connexion persistante

1. Une fois connecté, dans le champ de message, envoyez le texte `Bonjour serveur`.
2. Observez la réponse : le serveur d'écho vous renvoie exactement le même message.
3. Envoyez un second message, différent, **sans vous reconnecter**.
4. Observez à nouveau la réponse.

Question à consigner : entre l'envoi du premier et du second message, avez-vous dû rouvrir une connexion ou renvoyer une quelconque forme d'authentification d'URL ? Qu'est-ce que cela démontre concrètement à propos du caractère « persistant » d'une connexion WebSocket, par opposition au modèle requête-réponse classique de REST ?

Tâche 3 — Observer les limites

Fermez l'onglet Postman contenant votre connexion WebSocket (ou cliquez sur **Disconnect**), puis rouvrez une nouvelle connexion.

Question à consigner : que devient l'historique des messages échangés lors de la session précédente ? Dans le projet de synthèse (messagerie), comment avez-vous prévu de contourner cette limite pour l'historique des conversations ?

Partie 2 — Server-Sent Events (SSE)

Tâche 4 — Observer un flux SSE réel

Nous utilisons `Wikimedia EventStreams`, un service public officiel de la fondation Wikimedia qui diffuse en continu, au format SSE, toutes les modifications faites sur Wikipédia et ses projets frères.

1. Ouvrez un terminal et exécutez :

```
curl -N https://stream.wikimedia.org/v2/stream/recentchange | head -n 15
```

(L'option `-N` désactive la mise en tampon de curl, indispensable pour voir les événements arriver au fur et à mesure plutôt que d'un bloc à la fin ; `head -n 15` arrête la commande après 15 lignes. Le flux étant très actif, des événements arrivent plusieurs fois par seconde. Dans un navigateur, vous pouvez aussi ouvrir directement l'URL ci-dessus.)

2. Observez les lignes qui apparaissent : chaque événement est précédé de lignes `event:` et `id:`, puis d'une ligne `data:` contenant un objet JSON. Si le service est inaccessible depuis le réseau de l'école, passez directement à la Tâche 5 et utilisez votre propre endpoint comme source de flux.

Question à consigner : identifiez, dans la sortie, la structure `data: ...` vue en cours (Section 5.2 du Chapitre 6). Cette connexion s'est-elle fermée après le premier événement reçu ?

Tâche 5 — Construire votre propre endpoint SSE avec Spring Boot

Objectif : passer de l'observation à la construction.

1. Dans votre projet Spring Boot, créez un contrôleur `HorlogeController` :

```java
package com.example.tp.controller;

import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
import java.time.LocalTime;
import java.util.concurrent.Executors;

@RestController
public class HorlogeController {

    @GetMapping("/horloge")
    public SseEmitter streamHorloge() {
        SseEmitter emitter = new SseEmitter(0L); // pas de timeout
        Executors.newSingleThreadExecutor().execute(() -> {
            try {
                for (int i = 0; i < 20; i++) {
                    emitter.send("Heure actuelle : " + LocalTime.now());
                    Thread.sleep(1000);
                }
                emitter.complete();
            } catch (Exception e) {
                emitter.completeWithError(e);
            }
        });
        return emitter;
```

```
    }
}
```

2. Lancez l'application et ouvrez `http://localhost:8080/horloge` **directement dans votre navigateur** (pas Postman, pour cette étape — les navigateurs affichent nativement un flux `text/event-stream`).
3. Observez l'heure défiler automatiquement dans la page, sans aucun rechargement.

Question à consigner : dans ce code, `SseEmitter(0L)` désactive le timeout par défaut. Que se passerait-il, côté utilisateur, si ce timeout était atteint en plein milieu du flux ? Reliez votre réponse à l'avantage de SSE vu en cours (Section 5.3) concernant la reconnexion.

Partie 3 — Webhooks

Tâche 6 — Inspecter un Webhook en temps réel avec webhook.site

1. Ouvrez `https://webhook.site` dans votre navigateur. Une URL unique vous est automatiquement attribuée (ex. `https://webhook.site/xxxxxxxx-xxxx-...`). **Gardez cet onglet ouvert.**
2. Dans Postman, créez une requête `POST` vers **cette URL**, avec le corps JSON suivant :

```json
{
  "evenement": "paiement.reussi",
  "montant": 49.90,
  "transactionId": "tx_8827"
}
```

3. Envoyez la requête, puis retournez immédiatement sur l'onglet `webhook.site`.

Question à consigner : la requête envoyée depuis Postman apparaît-elle instantanément sur `webhook.site`, sans que vous n'ayez rien rafraîchi ? En quoi cette page elle-même utilise-t-elle une des deux technologies vues en Partie 1 ou 2 de ce TP pour se mettre à jour en direct ?

Tâche 7 — Décomposer la requête reçue

Sur `webhook.site`, cliquez sur la requête que vous venez d'envoyer.

Question à consigner : retrouvez, dans les détails affichés, les éléments suivants vus en cours (Section 6.3 du Chapitre 6) :

1. La méthode HTTP utilisée.
2. Les en-têtes envoyés (dont `Content-Type`).
3. Le corps JSON, tel qu'il a été reçu.

Confirmez, à partir de ce que vous observez, la phrase du cours : *« un Webhook n'est, dans sa structure, qu'une requête HTTP tout à fait ordinaire »*.

Partie 4 — Streaming d'événements à grande échelle (exercice de conception)

Tâche 8 — Concevoir une architecture pub/sub

Scénario : reprenez la bibliothèque en ligne des TP précédents. Trois services internes doivent tous réagir chaque fois qu'un livre est emprunté :

- Le service **Catalogue** (pour marquer le livre comme indisponible).
- Le service **Notifications** (pour prévenir les utilisateurs ayant mis ce livre en liste d'attente).
- Le service **Statistiques** (pour mettre à jour les tableaux de bord de popularité).

Travail à faire :

1. Proposez un nom de topic Kafka pour cet événement, sur le modèle vu en cours (Section 7.3 du Chapitre 6).

2. Lequel de ces trois services est un **producteur**, lesquels sont des **consommateurs** ?
3. Un quatrième service, de recommandation de livres, est ajouté six mois plus tard. Expliquez, à partir du principe de découplage pub/sub vu en cours, pourquoi son ajout ne nécessite **aucune modification** du service qui publie l'événement d'emprunt.
4. Le service Statistiques tombe en panne pendant deux heures. Que se passe-t-il pour les événements publiés pendant cette période, si l'on utilise Kafka plutôt qu'un simple Webhook vers ce service ? Justifiez avec la propriété de persistance vue en cours.

Rapport et soumission

Votre rapport doit inclure :

1. Une capture d'écran de votre connexion WebSocket (Tâches 1-2).
2. La sortie de la commande `curl` de la Tâche 4.
3. Une capture d'écran de votre endpoint `/horloge` fonctionnant dans le navigateur (Tâche 5).
4. Une capture d'écran de la requête reçue sur `webhook.site` (Tâches 6-7).
5. Les réponses rédigées à toutes les questions « à consigner ».
6. Vos réponses à la Tâche 8 (conception pub/sub).

Déposez votre rapport au format PDF sur la plateforme du cours.

Barème indicatif

Critère	Points
Partie 1 — WebSockets (Tâches 1 à 3)	5
Partie 2 — SSE (Tâches 4 et 5)	6
Partie 3 — Webhooks (Tâches 6 et 7)	5
Partie 4 — Conception pub/sub (Tâche 8)	3
Qualité du rapport	1
Total	**20**